

# CSP-J 知识自查表

## 面向 OI 实战的知识、复杂度与实现风险检查

示例基线：C++14 (-O2 -std=c++14)，具体语言版本、时限和内存以当年比赛公告为准。

**自查标准：**每一行至少能说出适用条件、手写正确实现、估算复杂度，并构造一个能击穿错误写法的边界或反例。

## 目录

|                      |    |
|----------------------|----|
| 1 C++ 语言与实现基础        | 2  |
| 2 基本问题求解与复杂度         | 3  |
| 3 字符串                | 4  |
| 4 数据结构、搜索与图论         | 5  |
| 5 动态规划               | 6  |
| 6 数学                 | 7  |
| 7 经典算法复杂度速查          | 9  |
| 8 数据范围与复杂度反推 (OI 视角) | 10 |
| 9 C++ UB 与高频实现风险     | 12 |
| 10 对拍流程              | 13 |
| 11 常见缺口              | 15 |
| 创作声明与许可              | 15 |

# 1 C++ 语言与实现基础

| 知识点                | 什么时候用                                                                                                                          | 怎么检查自己                                                                                                                                                |
|--------------------|--------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| 整数类型范围溢出           | 写任何可能产生大数的运算时：答案会否超过 $2^{31} - 1$ ？乘法先溢出再赋值给 long long 就晚了。                                                                    | 写乘方：输入 $a, b$ ( $a, b \leq 10^9$ )，若 $a^b > 10^9$ 输出 $-1$ 。想清楚为什么不能先乘再判断。                                                                             |
| 整数除法与取模            | 整除、向上取整、余数状态和二进制拆分。C++14 的有符号整数除法向零取整；负数取模的符号也要按语言规则判断。                                                                        | 给定 $a = -7, b = 3$ ，输出 C++14 下 $a/b$ 和 $a\%b$ ，解释结果差异。                                                                                                |
| 浮点误差与精确比较          | 百分比、 $\sqrt{\Delta}$ 、分数比较：优先整数。double 无法精确表示 0.1。                                                                             | 写整数二分开方 $\text{isqrt}(n)$ ，禁止用 $\text{sqrt}$ 。用 $n = 10^{18}$ 验证。                                                                                     |
| 数组、字符、string       | 读含空格的句子、解析 IP、统计字符频次。cin >> s 会跳过空格。                                                                                           | 读一整行（可能含空格和数字），统计字母和数字各出现几次。                                                                                                                          |
| 函数、引用、指针           | 传递大型对象（如 vector、邻接表）时避免不必要复制；小型标量按值传递未必更慢。                                                                                     | 将 <code>void f(vector&lt;int&gt; v)</code> 改为 <code>void f(const vector&lt;int&gt;&amp; v)</code> ，解释快的原因。                                            |
| 递归与调用栈             | 树遍历、分治排序、表达式求值用递归最直接；深度到 $10^5$ 量级可能爆栈。栈帧大小和默认栈上限取决于编译器、平台和局部变量，不能用固定字节数估算。                                                    | 写一个 DFS 遍历树（递归版），再改成显式栈版。说明何时必须改。                                                                                                                     |
| 输入输出与文件 IO         | 输入输出量大时可用 <code>std::ios::sync_with_stdio(false)</code> 与 <code>std::cin.tie(nullptr)</code> ；是否超时取决于数据量、输出量和时限，不存在固定的 $n$ 阈值。 | 用 <code>freopen</code> 重定向的例子。解释 <code>cin &gt;&gt;</code> 后接 <code>getline</code> 读到空串的原因。                                                           |
| vector、queue、deque | 邻接表、BFS 队列、模拟队列。分清各容器适用场景。                                                                                                     | 手写循环队列（push/pop/front，用数组），说出 <code>std::queue</code> 底层是 deque。                                                                                      |
| STL map、set、heap   | 值域太大不能开数组、需按序遍历、Dijkstra 取最小距离。默认 <code>priority_queue</code> 是大根堆。                                                            | 写 <code>priority_queue</code> 最小堆版本，用它跑 Dijkstra。                                                                                                     |
| 运算符优先级、结合性与短路      | 写表达式解析或混合多个运算符时，不能靠猜优先级。位运算 $\& \mid \sim$ 优先级低于 $== \neq$ ；逻辑 $\&\& \mid\mid$ 有短路语义；赋值 = 优先级极低。括号是最便宜的正确性保障。                  | 给出 <code>int a=2,b=3,c=4;</code> ，不加括号解释 <code>a&amp;b==2</code> 和 <code>a b&amp;c</code> 的实际求值顺序，并说明为何 <code>if(x&amp;1==0)</code> 不判断 $x$ 最低位是否为 0。 |
| C++14 版本与编译选项      | 本文以 <code>-std=c++14</code> 为基线； <code>std::gcd</code> 、结构化绑定、 <code>if constexpr</code> 属于 C++17，不能依赖编译器扩展。                   | 用 <code>g++ -std=c++14 -O2</code> 编译最近写的一份代码；静态链接、编译器和内存限制以比赛环境为准。                                                                                    |

## 2 基本问题求解与复杂度

| 知识点           | 什么时候用                                                        | 怎么检查自己                                                                        |
|---------------|--------------------------------------------------------------|-------------------------------------------------------------------------------|
| 模拟            | T1 或部分分常见。按题面一步一步做即可，难点在别漏条件。                                | 把题面描述拆成三条不可交换的”每次循环做啥”的步骤。                                                    |
| 枚举            | 数据规模小的时候枚举所有可能性；剪枝只能降低实际运行量，不能改变最坏复杂度。                       | 写 $N$ 个数的子集枚举（位运算版和递归 DFS 版各一）。                                               |
| 递归、分治、显式栈     | 分治排序、树遍历、括号解析。递归写起来快，深度大改迭代。                                 | 写归并排序递归版和迭代版，说出两个版本的递归深度和栈空间差异。                                               |
| 贪心            | 单向加油、区间选最多、最早结束最有利。 $O(N)$ 或 $O(N \log N)$ ，但必须有证明。          | 想一个”显然贪心”的题，构造反例证明你错了。                                                        |
| 贡献计算          | 把总答案拆成每个元素、位置、边或区间的独立贡献，常能把两重枚举改成一次统计、前缀和或频次维护。              | 求所有子数组元素和时，推导 $a_i$ 被多少个子数组包含；再用小规模暴力对拍贡献式。                                   |
| 前缀和、差分        | 前缀和适合频繁区间聚合查询；差分适合区间加减，最后扫描一次还原。两者的 $O(1)$ 是针对特定操作，不是所有区间操作。 | 实现数组区间加 1、最后输出所有位置的值，要求 $O(N + Q)$ 。                                          |
| 二分查找与二分答案     | 有序数组中找边界，或答案范围大（如 $10^9$ ）且判定具有单调性；先区分找任意值、第一处真和最后一处真。       | 分别写 <code>lower_bound</code> 、第一处可行和最后处可行模板，处理空区间、重复值和中点溢出。                   |
| 三分查找          | 目标函数在区间内单峰或单谷，且能比较两个位置的函数值时使用；不满足单峰性不能套模板。                   | 在整数区间上写三分并处理平台、端点和最终枚举；说明实数三分的精度停止条件。                                         |
| 双指针、滑动窗口、单调队列 | 时间窗（24 小时内乘客）、价格窗（45 分钟票）、坐标窗口：窗口单向滑动。                       | 证明”每个元素最多出入队一次”。举看似两重循环、实为 $O(N)$ 的例子。                                        |
| 离散化           | 值域 $10^9$ 但有效值只有 $10^5$ 个，直接开数组会爆内存。                         | 整数数组排序去重后用 <code>lower_bound</code> 做压缩映射，写出完整代码。                             |
| 倍增            | 树上跳 $k$ 级祖先、数组跳区间。把 $k$ 按二进制拆位。                              | 给出 $up[j][v]$ 的两种含义：从 $v$ 跳 $2^j$ 步到哪，以及转移方程。                                 |
| 复杂度分析         | 别只看循环嵌套层数，还要看每个元素被处理的总次数。队列/栈常用”每个元素进出一次”摊销。                 | 给一段同时含 <code>while</code> 弹队列和 <code>for</code> 扫描的代码，判断 $O(N)$ 还是 $O(N^2)$ 。 |
| 部分分与测试点       | 若题面提供测试点或分组限制，小规模、全相等、 $k=0$ 等条件可能对应部分分；没有测试点表时也要根据约束自行分层。   | 看一道新题的约束与测试点，标出哪些子任务能用暴力、哪些需要优化。                                              |
| 边界数据          | 最小/最大/全相等/无解/两端相等：样例没有，必须自己造。                                | 为一段排序代码造 10 组数据，覆盖升序、降序、全相等、含负数、单元素。                                          |
| 正确性说明         | 写循环前想好”跑完后这个变量代表什么”，能抓大半 bug。                                | 给每个核心循环补注释：进入前和退出后，某个变量代表什么。                                                  |

### 3 字符串

| 知识点          | 什么时候用                                                                                                           | 怎么检查自己                                                                                                                                                                                             |
|--------------|-----------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 字符串解析与格式转化   | 进制转换（读入 $k$ 进制字符串转整数、或整数转 $k$ 进制输出）、高精度读入（逐字符转数字数组）、IPv4/端口格式校验（a.b.c.d:e 分五段，禁前导零、检查值域）。核心是逐字符处理、手工分段、用整数数组承接。 | 写 $k$ 进制转十进制：逐位累加 $\text{ans} = \text{ans} * k + \text{digit}$ 。写高精度读入：把长度 $\leq 10^6$ 的数字串转成 <code>vector&lt;int&gt;</code> 。写 IPv4 解析：严格拒绝空段、前导零（如 01）、越界（ $0 \sim 255$ 和 $0 \sim 65535$ ）和多余字符。 |
| 字符串暴力与部分分    | CSP-J 的字符串题往往 $n \leq 1000$ ， $O(N^2)$ 的暴力扫描（逐对比较子串、逐查询扫描全表）就能拿大量分数甚至满分。不要一上来就想高级结构。                            | 写“后缀查询”：有 $n$ 个书号、 $q$ 个需求码，每次查询扫描所有书号，用 $\text{code} \% 10^{\text{len}} == \text{need}$ 判后缀，复杂度 $O(nq)$ 在 $n, q \leq 1000$ 时完全可过。对比自己是否第一反应去写 Trie。                                               |
| 回文判断、子串与字符频次 | 回文： $O(N)$ 双指针或构造候选年份。子串计数：滑动窗口或枚举起点。字符频次：开 26 或 256 的数组统计即可。这些都是 CSP-J T1/T2 的高频考点。                            | 写回文日期题：枚举 4 位年份按回文规则构造 8 位日期，验证月日合法性后判断是否在输入区间内。再写标题统计题：逐字符分类计数字母和数字，注意空格和换行。说明“枚举年份构造”和“逐日推进”的复杂度差异。                                                                                              |
| Trie（前缀树）    | 大量字符串、前缀查询；即使不直接裸考，也可能作为字典、前缀统计或状态结构的组件。                                                                        | 用数组写 Trie，插入 cat/car/dog，查前缀 ca 的词数。                                                                                                                                                               |
| 字符串哈希        | 快速比较子串、判断树镜像。预处理 $O(L)$ 后每次查询 $O(1)$ ，但哈希存在碰撞风险。                                                                | 约定区间为半开区间 $[l, r)$ ，推导 $\text{hash}(l, r) = H[r] - H[l] \times \text{base}^{r-l}$ ；说明模数、底数和双哈希如何降低风险。                                                                                              |

## 4 数据结构、搜索与图论

| 知识点                | 什么时候用                                                                | 怎么检查自己                                                                                                                    |
|--------------------|----------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| 栈、队列、双端队列          | BFS ( 队列 )、括号匹配 ( 栈 )、滑动窗口最大值 ( 单调队列 )。元素最多进出一次是摊销核心。                | 手写循环队列 ( 数组实现 , 不用 STL ), 解释判空判满为什么要留一个空位。                                                                                |
| 堆与优先队列             | Dijkstra 取最小距离、事件模拟最早发生。默认大根堆, 取最小值得改比较器。                            | 写 Dijkstra: 为什么同一点可能入堆多次? 旧的必须跳过?                                                                                         |
| 集合、映射、哈希           | 值域小用数组; 需要有序遍历或最坏复杂度保证时用 map; 能接受期望 $O(1)$ 且注意退化风险时再用 unordered_map。 | 构造一组让 unordered_map 退化成 $O(N)$ 的输入。                                                                                       |
| 多重计数与值映射           | 重复值按频次、类别或出现位置统计; 值域小用频次数组, 值域大用 map、哈希或离散化, 不能把”不同值个数”和”元素总数”混为一谈。  | 对 [3, 1, 3, 2, 3] 同时输出每个值的频次、出现位置和不同值个数, 比较数组、map 与离散化三种写法。                                                               |
| 并查集                | 只问”两个点是否连通”, 不问路径。                                                   | 手写 find ( 路径压缩 ) 和 merge ( 按大小合并 ), 说明为什么路径压缩不破坏正确性。                                                                      |
| 树与二叉树              | 树的遍历、子树大小、镜像判断。空子树怎么表示、递归迭代防爆栈。                                      | 用数组存二叉树, 写先序和后序遍历, 禁止递归。                                                                                                  |
| 图存储与遍历             | 稀疏图或点数较大时用邻接表; 很小的稠密图才考虑邻接矩阵。有向图、无向图建图不同。                            | 给一张 $N$ 点 $M$ 边无向图, 用邻接表, BFS 求 1 到各点最短步数。                                                                                |
| 链表与有序块删除           | 反复取走序列元素、合并相邻同类块。链表在已经定位节点时删除可为 $O(1)$ , 查找节点和维护块边界仍可能成为瓶颈。          | 模拟 0011100: 每轮取每个块最左元素, 写出每轮输出。                                                                                           |
| 树状数组               | 单点修改、前缀和查询。比线段树短。稳定排序里的排名也能算。                                        | 手写 add 和 sum, 说明 lowbit。在 [3, 1, 3] 上模拟全部更新。                                                                              |
| DFS、BFS            | 可达性、无权最短路、连通块。标记要在入队时做, 而非出队时。                                       | 在有环图上逐层写出 BFS 入队顺序, 解释为何标记时机重要。                                                                                           |
| Dijkstra 与 0-1 BFS | 边权非负时可用 Dijkstra; 边权仅为 0, 1 时可用 deque 做 0-1 BFS。一般非负权不应误写成普通 BFS。    | 用天平类比松弛: 为什么非负权下当前弹出的最小有效距离可以定型? 旧堆项为什么必须跳过?                                                                              |
| 状态图 ( 奇偶/资源维 )     | ”恰好 $L$ 步”、”最多 $k$ 次魔法”、”必须模 $k$ 时刻”: 加维度进状态。                        | 无向图 $s$ 到 $t$ 恰好 $L$ 步不能只看最短路; 还要考虑连通分量、二分图奇偶性和是否存在可往返的边或奇环。构造二分图和含奇环的图, 分别判断恰好 $L$ 步可达性, 说明为什么”来回 2 步”只是常用的扩展手段, 不是完整判定。 |

## 5 动态规划

| 知识点             | 什么时候用                                                                                                                                                                                                                                                 | 怎么检查自己                                                                                                                     |
|-----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| 状态定义、转移、初始化     | 写 DP 先定状态： $dp[i]$ 是“以 $i$ 结尾”还是“前 $i$ 个中选”。初始化不能默认 0；“恰好”通常用 $-\infty$ 标不可达，“至多”或空集合法可用 0。                                                                                                                                                           | 写“恰好 $n$ 根拼数字”的 DP，解释不可达为何不能初始化为 0。再写“网格取数”：允许上下右移动、格权可为负，全负网格为何 DP 初值不能是 0 而要用 $-\infty$ 。                                |
| DP 识别：从暴力到 DP   | 题面要求“最多/最少/方案数”、数据 $n \leq 500$ 但 $2^n$ 枚举不可能时，优先考虑 DP。核心信号：存在重叠子问题（同一状态被多次计算）、存在最优子结构（大问题的最优解包含子问题的最优解）。                                                                                                                                             | 给摆渡车问题： $n$ 个同学在不同时刻到达，发车间隔固定为 $m$ 。先写按到达时间分组的 $O(2^n)$ 暴力枚举发车方案，再指出“前 $i$ 个人的最小等待和”只依赖前面状态，写出 $dp[i]$ 转移。对比暴力与 DP 的复杂度差距。 |
| 背包 DP           | 选若干物品使和满足某条件。对正权容量的一维 DP，0/1 背包通常从大到小，完全背包通常从小到大；计数、负权或其他转移要重新证明顺序。                                                                                                                                                                                   | 写 0/1 背包，解释容量循环方向，再改成完全背包。                                                                                                 |
| 网格 DP / 带预算 DP  | 网格上单向走、带预算 $k$ 选点。1000 × 1000 时不能 $O(N^3)$ 。预算维通常来自题目的 $k \leq 100$ 这类小参数信号。                                                                                                                                                                          | 3 × 4 网格手动跑 DP，只准右/下，取最大和。再写出状态为 $dp[i][j][used]$ 的三维带预算版。                                                                 |
| 线性 DP 与区间 DP 入门 | 线性 DP ( $dp[i]$ 只依赖前若干项) 是 CSP-J 最常考的 DP 形态。区间 DP ( $dp[l][r]$ 依赖子区间) 在回文、括号、取数等场景出现。两者都可用 $O(N^2)$ 拿部分分，再优化。                                                                                                                                         | 写最长上升子序列 $O(N^2)$ 版。再写“网格取数”的按列 DP：每列从上到下、从下到上各扫一次，解释为何不能在同一列内来回走（违反不能重复经过的约束）。                                            |
| 单调队列优化 DP       | 转移从滑动窗口取 max/min 时，直接枚举多 $O(W)$ ，单调队列压到 $O(N)$ 。                                                                                                                                                                                                      | 数组 [5, 1, 4, 4, 2]，窗口 3，逐步写出队列变化。                                                                                          |
| 方案恢复与滚动数组       | 只问最优值时可滚动；要恢复方案时通常保存父指针或决策，不能把必要信息一起覆盖。也可用重算等方法换空间。                                                                                                                                                                                                   | 在背包 DP 上加 choice 数组，输出最优值的同时输出选了哪些物品。                                                                                      |
| DP 维度选择与部分分信号   | 从题面约束倒推 DP 维数： $n \leq 500$ 且 $k \leq 100 \rightarrow$ 二维 $dp[i][k]$ （带预算的点列选择）； $m \leq 100$ 且 $n \leq 500 \rightarrow dp[i]$ 一维 + 余数优化（固定间隔发车）；网格 1000 × 1000 $\rightarrow$ 按列滚动（网格取数）；值域小 ( $a_i \leq 5000$ ) $\rightarrow$ 以值域为容量做背包 DP（选木棍拼多边形）。 | 给一道新题的数据范围，能写出至少两种可能的 DP 状态设计，并估算各自的时间/空间复杂度，选出更可行的方案。                                                                     |

## 6 数学

| 知识点            | 什么时候用                                                                                                                                                                                                                                                                      | 怎么检查自己                                                                                                                                                   |
|----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| 基本运算与边界        | max/min、绝对值。abs(INT_MIN) 的结果无法由 int 表示，取负或调用对应重载可能触发未定义行为。                                                                                                                                                                                                                 | 不用分支和库函数写整数绝对值。说出 abs(INT_MIN) 的后果。                                                                                                                      |
| 位运算            | 判奇偶、提取二进制位、异或 ( $a \oplus a = 0$ )、lowbit、枚举子集 (for(int s=mask; s; s=(s-1)&mask))。                                                                                                                                                                                         | 写代码：对正整数只用位运算判 2 的幂，不准用循环；用位运算枚举 $\{a, b, c\}$ 的所有非空子集。                                                                                                  |
| 质数判定与埃氏筛       | 单个数判质数可试除到 $\sqrt{n}$ ；需要批量筛一段连续范围的质数时用埃氏筛， $O(N \log \log N)$ 。                                                                                                                                                                                                           | 手写埃氏筛求 $[1, 100]$ 的质数表，说明为什么内层从 $i^2$ 开始、为什么只用 $\leq \sqrt{N}$ 的质数去筛。                                                                                    |
| 线性筛（欧拉筛）与最小质因子 | 要求 $O(N)$ 或需要每个数的最小质因子时用线性筛。核心：每个合数只被其最小质因子筛掉一次。可顺势维护 $lp[x]$ 用于 $O(\log x)$ 因数分解或积性函数递推。                                                                                                                                                                                  | 写线性筛同时维护 prime[] 和 $lp[x]$ ，在纸上模拟 $\{2 \dots 30\}$ ：指出 12 被 2 筛、15 被 3 筛、25 被 5 筛，说明判断 $i \% prime[j] == 0$ 后 break 的原因。                                 |
| gcd 与 lcm      | 约分、分数化简。std::gcd 是 C++17 才有，C++14 自己写；计算 lcm 时先除后乘以降低溢出风险。                                                                                                                                                                                                                 | 手算 gcd(12345, 67890)，写每一步余数。                                                                                                                             |
| 模运算与快速幂        | 答案模 $10^9 + 7$ 或 998244353；指数很大时用快速幂把乘法次数从 $O(n)$ 降到 $O(\log n)$ 。                                                                                                                                                                                                         | 写快速幂递归和迭代版各一，验证 $3^{10} \bmod 7 = 4$ ，并说明二分拆指数的正确性。                                                                                                      |
| 贡献计算           | 不枚举所有子结构，改为计算每个元素在最终答案中被计入的次数。常用于“所有子数组的 max/min/sum 之和”“每个节点对多少条路径有贡献”等聚合题。核心公式：<br>$ans = \sum_i val_i \times count_i。$                                                                                                                                                  | 给定数组 $[2, 1, 3]$ ，用单调栈求每个元素作为最大值的左右边界，计算以它为最大值的子数组个数，验证所有子数组最大值之和是否为 $2 \cdot 1 + 1 \cdot 1 + 3 \cdot 3 = 12$ 。手算与代码一致。                                  |
| 组合数学基础         | 先判断分类是否互斥且完备，再用加法原理；分步完成且步骤相互独立时用乘法原理。计数题不要一上来就写 DP。                                                                                                                                                                                                                       | 对“选一个人”与“先选组别再选组内成员”各写一种计数式，指出重复计数和漏计数的位置。                                                                                                               |
| 排列、组合与重复计数     | 区分排列 $P(n, k)$ 、组合 $C(n, k)$ 、可重复选择 ( $n^k$ ) 和相同元素排列（多重集排列公式）。常与全排列、子集、频次分配结合。                                                                                                                                                                                            | 手算 $C(10, 3)$ ，分别解释“从 10 人中选 3 人排队”和“选 3 人开会”的答案为何不同。给 $\{a, a, b\}$ 写出所有不同排列。                                                                           |
| 组合递推与杨辉三角      | $C(n, k) = C(n-1, k-1) + C(n-1, k)$ ，边界为 $C(n, 0) = C(n, n) = 1$ ；大量询问时预处理、取模、滚动空间。                                                                                                                                                                                        | 写杨辉三角前 5 行，验证一行的和为 $2^n$ ，并说明组合数取模时不能随意使用实数除法（要用递推或乘法逆元）。                                                                                                |
| 多重计数、映射与双射     | 直接计数困难时，建立原问题与另一个易计数集合的双射（一一对应）；或按某个特征分类后对每类分别计数再相加；也可用“计数 $\times$ 量 = 总量”的双视角技巧。                                                                                                                                                                                         | 用隔板法（Stars and Bars）解释“长度为 $n$ 、和为 $S$ 的非负整数序列个数”为 $C(S+n-1, n-1)$ ，手算 $n=3, S=4$ 的结果并列验证。举例说明如何把“子数组和为 $K$ ”映射为前缀差对 $(i, j)$ 满足 $pre[i] - pre[j] = K$ 。 |
| 一元幂方程与单调方程     | 求 $x^k = A$ 、 $ax + b = c$ 或单调函数的整数根时，利用定义域、单调性和二分；一般高次多项式没有可直接套用的通用初等公式。                                                                                                                                                                                                  | 写整数二分求最大的 $x$ 使 $x^k \leq A$ ，避免乘法溢出并回代检查；比较整数答案和实数近似的处理差异。                                                                                              |
| 一元二次方程求解       | $ax^2 + bx + c = 0$ ( $a \neq 0$ )：先算 $\Delta = b^2 - 4ac$ 。 $\Delta < 0$ 无实根； $\Delta \geq 0$ 继续。整数根条件： $\Delta$ 为完全平方数且 $-b \pm \sqrt{\Delta}$ 被 $2a$ 整除。非整数需精确输出：约分有理数，或提取 $\sqrt{\Delta}$ 的平方因子写成规范根式 $\frac{p \pm q\sqrt{s}}{r}$ ，保证 $\gcd(p, q, r) = 1$ 且 $s$ 无平方因子。 | 写程序对 $x^2 - 3x + 1 = 0$ 输出两个根的规范形式（含 $\sqrt{5}$ ），并处理 $a < 0$ 、 $b^2$ 溢出（用 long long）、 $\Delta$ 非平方、分母约分和较大根选择。对照历年真题中 RSA 解密和一元二次方程输出题验证。               |
| 整数开方           | 需要精确整数答案或判完全平方数时，优先用整数运算并验证；不能把浮点 sqrt 的结果未经检查直接转整数。                                                                                                                                                                                                                       | 写整数二分开方，避免 mid*mid 溢出，验证 $\lfloor \sqrt{10^{18}} \rfloor$ 。                                                                                              |

| 知识点   | 什么时候用                                                                                                                                                                      | 怎么检查自己                                                                                           |
|-------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|
| 二分与三分 | 二分用于单调函数求零点或阈值：整数域上用闭区间不变量防死循环，实数域按精度或固定次数迭代。三分用于单峰/单谷函数求极值：每轮比较 $f(m_1)$ 与 $f(m_2)$ 扔掉极值不在的 $1/3$ 区间；离散域上区间缩小到 $\leq 3$ 后枚举。与第 2 节“二分答案”的差异：这里面对的是具体函数 $f(x)$ ，不要求构造判定谓词。 | 写整数三分在 $[0, 10]$ 上找 $f(x) = -(x-3)^2 + 5$ 的最大值，先用枚举验证极值位置，再解释三分为何每次扔掉 $1/3$ 区间，并说明整数三分何时需要转枚举收尾。 |



## 7 经典算法复杂度速查

| 算法                            | 时间复杂度                             | 空间复杂度               | 实战提醒                                        |
|-------------------------------|-----------------------------------|---------------------|---------------------------------------------|
| 线性扫描 / 模拟                     | $O(N)$                            | $O(1)$ 或 $O(N)$     | $N \leq 10^6 \sim 10^7$ 只是粗略量级；单步若很重仍可能超时。  |
| 贡献计算 / 频次统计                   | 常见 $O(N)$ ，排序或映射后 $O(N \log N)$   | $O(N)$ 或 $O(A)$     | 先证明每个对象被统计恰好一次；重复值、值域大小和答案溢出都要单独检查。         |
| 排序 ( <code>std::sort</code> ) | $O(N \log N)$                     | 通常 $O(\log N)$ 栈    | 复杂度保证和常数都要看；比较器必须满足严格弱序。                    |
| 二分答案 + 判定                     | $O(\log R \times T(N))$           | 取决于判定               | 判定必须单调；不能因为二分只有 $\log R$ 次就忽略 $T(N)$ 。      |
| 三分查找                          | $O(\log R \times T(N))$ 或按精度迭代    | $O(1)$ 或判定所需空间      | 只适用于单峰/单谷函数；整数三分收缩到小区间后要枚举，不能把二分的单调谓词条件混过来。 |
| 前缀和 / 差分                      | 预处理 $O(N)$ ，查询或更新可 $O(1)$         | $O(N)$              | 前缀和需能用减法等还原；差分的区间更新后还要扫描还原。                 |
| 双指针 / 滑窗                      | $O(N)$                            | $O(N)$ 或 $O(1)$     | 只有端点单向移动且总移动次数可摊销时成立。                       |
| 单调队列 / 单调栈                    | $O(N)$ 摊销                         | $O(N)$              | 每个元素至多入队（栈）一次、被弹出一次；窗口或单调性必须成立。             |
| BFS / DFS                     | $O(V + E)$                        | $O(V + E)$ 总计       | 这里的空间包含邻接表；若图已存好，额外队列、栈和标记通常是 $O(V)$ 。      |
| Dijkstra (堆)                  | $O((V + E) \log V)$               | $O(V + E)$          | 需要非负边权；懒删除堆项要跳过。                            |
| 0-1 BFS                       | $O(V + E)$                        | $O(V + E)$ 总计       | 仅适用于边权 $\in \{0, 1\}$ ，不是所有小整数边权都能直接套用。     |
| 埃氏筛 / 线性筛                     | $O(N \log \log N)$ / $O(N)$       | $O(N)$              | 线性筛通常同时维护质数表和最小质因数；数组下标、0/1 和合数标记边界必须统一。    |
| Floyd-Warshall                | $O(V^3)$                          | $O(V^2)$            | $V = 400$ 已有 64M 次量级的松弛，是否能过要结合时限和常数实测。     |
| 并查集                           | 单次均摊 $O(\alpha(N))$               | $O(N)$              | 支持连通性合并；不直接提供路径，常用于 Kruskal。                |
| Kruskal                       | $O(E \log E)$                     | $O(V + E)$          | 无向图最小生成树；需先按边权排序。                           |
| 拓扑排序                          | $O(V + E)$                        | $O(V + E)$          | 只有 DAG 才存在完整拓扑序，不能用来绕过有向环。                  |
| 树状数组                          | 单次 $O(\log n)$                    | $O(n)$              | 单点修改 + 前缀和；区间加法可用差分思想扩展。                    |
| 离散化                           | $O(M \log M)$                     | $O(M)$              | $M$ 是收集到的坐标数，不是坐标数值；在线出现的新坐标不能事后补表。         |
| 组合计数 DP / 杨辉三角                | 常见 $O(NK)$ 或 $O(N^2)$             | $O(NK)$ 或滚动到 $O(K)$ | 先确认计数对象、顺序和是否允许重复；取模加法、乘法和答案类型都要检查。         |
| 字符串哈希                         | 预处理 $O(L)$ ，查询 $O(1)$             | $O(L)$              | 哈希有碰撞；双哈希能降低概率，但不能作为绝对证明。                   |
| Trie (数组)                     | 建树 $O(S)$ ，查询 $O(\text{len})$     | $O(S \Sigma )$      | $S$ 为所有字符串总长度；稀疏边存储可降低空间。                   |
| 朴素 DP                         | $O(S \times T)$                   | $O(S)$ 或压缩后更小       | 先数清状态数和每个状态的转移数，不能只看数组维数。                   |
| 背包 DP                         | $O(NC)$                           | $O(C)$ 滚动           | 0/1 和完全背包的循环方向只在相应转移条件下成立。                  |
| 单调队列优化 DP                     | $O(N)$ 摊销                         | $O(N)$              | 只有转移能化成窗口最值，且窗口端点单调时才成立。                    |
| 倍增                            | 预处理 $O(N \log R)$ ，查询 $O(\log R)$ | $O(N \log R)$       | 需要同一转移可重复合并，例如跳祖先或函数幂。                      |
| 高精度                           | 加减 $O(L)$ ，朴素乘法 $O(L^2)$          | 至少 $O(L)$           | 只有内建整数不够表示时才需要；位数、进位和输出都要计入。                |

## 8 数据范围与复杂度反推 (OI 视角)

拿到题先看测试点表。最小的  $n$  决定暴力能不能过，最大的  $n$  决定满分需要什么级别。

下面的数量是“最内层简单整数操作”的数量级，只用于初筛方案；这里先按单组数据总计约  $5 \times 10^8 \sim 10^9$  次极简单操作的量级估算。数组访存、字符串、取模、容器、函数调用、输入输出、测试点数量和时限都会改变常数；因此“能过”不能由一个  $O(\cdot)$  或一个  $n$  单独保证。

| 增长级别           | 代表性数量                                                                         | 只作初筛的范围                                   | OI 解释                                              |
|----------------|-------------------------------------------------------------------------------|-------------------------------------------|----------------------------------------------------|
| $n^n$          | $n \approx 8 \sim 9$ 时约<br>$10^7 \sim 4 \times 10^8$                          | 约 $n = 8 \sim 9$ ；上端需非常简单                 | 每个位置有 $n$ 种选择时出现；剪枝可能改变实际量，但不能默认最坏情况消失。            |
| $n!$           | $n \approx 11 \sim 12$ 时约<br>$4 \times 10^7 \sim 5 \times 10^8$               | 约 $n = 11 \sim 12$ ；上端需看常数                | 全排列、排列搜索常见；生成和复制一个排列也有额外代价。                        |
| $3^n$          | $n \approx 17 \sim 18$ 时约<br>$10^8 \sim 4 \times 10^8$                        | 约 $n = 17 \sim 18$ ；上端需非常简单               | 三进制状态、每个元素三种决策；转移若再乘一个 $n$ ，可接受范围还要缩小。             |
| $2^n$          | $n \approx 28 \sim 29$ 时约<br>$3 \times 10^8 \sim 5 \times 10^8$               | 约 $n = 28 \sim 29$ ；上端需很简单                | 状压 DP、子集枚举；转移若再乘一个 $n$ ，可接受范围还要缩小。                 |
| $n^5$          | $n \approx 45 \sim 50$ 时约<br>$2 \times 10^8 \sim 3 \times 10^8$               | 约 $n = 45 \sim 50$ ；上端需非常简单               | 五重循环只有在每层操作极轻、数据单组且常数较小时才考虑。                       |
| $n^4$          | $n \approx 120 \sim 150$ 时约<br>$2 \times 10^8 \sim 5 \times 10^8$             | 约 $n \leq 150$ ； $n \approx 150$ 时需看内层    | 简单数组整数循环有机会通过，但字符串、map、取模或多组数据可能超时，不能写成“稳过”。       |
| $n^3$          | $n \approx 400 \sim 500$ 时约<br>$6 \times 10^7 \sim 1.3 \times 10^8$           | 约 $n = 400 \sim 500$ ；上端需看常数              | Floyd、三维转移等； $n \approx 500$ 时已有约 $10^8$ 次松弛。      |
| $n^2$          | $n \approx (1.5 \sim 2) \times 10^4$ 时约<br>$2 \times 10^8 \sim 4 \times 10^8$ | 约 $n = (1.5 \sim 2) \times 10^4$ ；上端需简单内层 | 两重枚举和二维 DP 要同时检查常数、内存以及是否存在更小的有效规模； $10^5$ 时通常不可行。 |
| $n \log n / n$ | $n \approx 10^6 \sim 2 \times 10^6$ 时约为数千万次                                   | 约 $n = 10^6 \sim 2 \times 10^6$ ；上端需关注常数  | 排序、扫描、图遍历的常见目标；大输入仍要关注输入输出、内存访问和递归深度。              |

**读表原则：**先用数量级排除明显不合适的复杂度，再结合题面时限、空间和操作常数实测。比如  $n$  在  $120 \sim 150$  时， $O(n^4)$  已接近粗略上限，简单循环是否能通过仍取决于内层操作和实现常数。

| 数据规模或状态        | 优先检查的复杂度                  | 常见候选                   | 还要检查什么                                           |
|----------------|---------------------------|------------------------|--------------------------------------------------|
| 约 $n \leq 9$   | $n^n, n!, 3^n, 2^n$ 都可尝试  | 全排列、DFS、子集或三进制枚举       | 先算实际状态数；剪枝不能替代最坏复杂度，还要检查递归深度和状态去重。               |
| 约 $n \leq 12$  | $n!, 3^n, 2^n, n^5$       | 排列搜索、状态压缩、五重简单枚举       | $n^n$ 在 $n \approx 10$ 时已到 $10^{10}$ ，不能把它算入这一档。 |
| 约 $n \leq 18$  | $3^n, 2^n, n^5, n^4$      | 三进制状态、状压 DP、带剪枝搜索、低阶枚举 | $3^n$ 在上端已是数亿量级；转移若再乘 $n$ 或另一维，范围要缩小。            |
| 约 $n \leq 29$  | $2^n, n^5, n^4, n^3$      | 状压 DP、子集枚举、折半、低阶 DP    | $2^n$ 在上端已是数亿量级；状态转移和内存访问都要很简单。                  |
| 约 $n \leq 50$  | $n^5, n^4, n^3, n^2$      | 五重、四重或三重枚举，区间 DP       | $n^5$ 在上端已是数亿量级；内层是否连续访问数组、是否有多组测试很关键。           |
| 约 $n \leq 150$ | $n^4, n^3, n^2, n \log n$ | 四重枚举、Floyd、区间 DP、二维 DP | $n^4$ 在 $n \approx 150$ 时约为数亿次，需基准测试并控制常数。       |
| 约 $n \leq 500$ | $n^3, n^2, n \log n$      | Floyd、三重枚举、二维 DP、图论    | $n^3$ 在上端约为 $10^8$ 次；三重循环是否能过仍取决于内层和时限。          |

| 数据规模或状态                  | 优先检查的复杂度                                         | 常见候选                    | 还要检查什么                                                       |
|--------------------------|--------------------------------------------------|-------------------------|--------------------------------------------------------------|
| 约 $n \leq 2 \times 10^4$ | $n^2$ 、 $n \log n$ 或 $n$                         | 二维 DP、枚举点对、排序后扫描、树状数组   | $n^2$ 在上端约为数亿次，还要乘上测试组数、内层代价和空间开销。                           |
| 约 $n \leq 2 \times 10^6$ | $n \log n$ 或 $n$                                 | 排序、扫描、邻接表图算法、线性 DP      | $n \log n$ 在上端约为数千万次；还要检查常数、内存、输入输出和递归深度。                    |
| 值域 $A \leq 600$          | 视 $N$ 与 $A$ 的乘积决定                                | 桶计数、频次数组、 $O(N + A)$ 扫描 | 小值域不代表 $O(N \times A)$ 一定可行。                                 |
| 额外参数<br>$k \leq 100$     | $O(Nk)$ 或 $O(\text{状态} \times k)$                | 带预算 DP、模 $k$ 状态图        | 先算总状态数；多个小参数相乘也可能爆炸。                                         |
| 单个数值可达<br>$10^9$         | 仍可能是 $O(n)$ 扫描；按问题再看 $O(\log V)$ 或 $O(\sqrt{V})$ | 二分、数学公式、试除、二进制分解        | 大的是值域 $V$ ，不是元素个数 $n$ ；不能开 $V$ 大小数组，但逐个读 $n$ 个数仍然可以 $O(n)$ 。 |

**注意：**复杂度反推是经验工具，不是通过承诺。先写出能验证的暴力，再用测试点、约束和基准数据定位真正的瓶颈，通常比凭一句“ $n$  很小/很大”猜算法更可靠。

## 9 C++ UB 与高频实现风险

| 种类   | 风险                                          | 错误示例                                                     | 表现                      | 修正                                                     |
|------|---------------------------------------------|----------------------------------------------------------|-------------------------|--------------------------------------------------------|
| UB   | 有符号溢出                                       | <code>int x=2000000000; x+=x;</code>                     | 变负数；优化期认为此分支不执行         | 用 <code>long long</code> 或先判 <code>x&gt;LIMIT-a</code> |
| UB   | 左移溢出                                        | <code>1&lt;&lt;31</code> (32 位 <code>int</code> )        | 出人意料的值                  | <code>1LL&lt;&lt;31</code> 或 <code>1u&lt;&lt;31</code> |
| UB   | 数组越界                                        | <code>int a[10]; a[10]=0;</code>                         | 覆盖相邻变量、段错误              | 每次访问前检查下标                                              |
| UB   | 整数除零                                        | <code>int x=a/b; (b 可能为 0)</code>                        | 运行时未定义                  | 分母可能为零时先判                                              |
| UB   | 未初始化变量                                      | <code>int x; if(x){...}</code>                           | 结果随机                    | 定义时赋初值                                                 |
| UB   | 迭代器失效                                       | <code>v.erase(it); it++;</code>                          | 跳过元素或崩溃                 | <code>it=v.erase(it)</code>                            |
| UB   | 严格弱序不满足                                     | <code>sort(..., [](int a,int b){return a&lt;=b;})</code> | 排序崩溃                    | 改 <code>return a&lt;b;</code>                          |
| UB   | 悬空引用                                        | <code>auto&amp; r=*v.end();</code>                       | 非法内存                    | 不对 <code>end()</code> 解引用                              |
| UB   | 递归爆栈                                        | <code>void f(){f();}</code>                              | SIGSEGV                 | 设深度上限或改迭代                                              |
| UB   | C++14 求值顺序                                  | <code>arr[i]=i++;</code>                                 | 未定义行为 (C++14)           | 拆两行，先保存旧值                                              |
| UB   | <code>int</code> 乘后赋 <code>long long</code> | <code>long long x=a*b;</code>                            | 先在 <code>int</code> 中溢出 | <code>1LL*a*b</code>                                   |
| 数值   | 浮点判等                                        | <code>double a,b; if(a==b)</code>                        | 相等判不出                   | <code>fabs(a-b)&lt;eps</code> 或改整数                     |
| 非 UB | 哈希退化                                        | <code>unordered_map</code> 遇坏数据                          | 退化 $O(N)$ 超时            | 别当 $O(1)$ 最坏                                           |
| UB   | 基类析构非虚                                      | <code>Base*p=new Derived; delete p;</code>               | 通过基类删除是未定义行为            | 基类析构函数声明为 <code>virtual</code>                         |

**区分：**UB = 标准允编译器做任何事；未指定 = 标准没规定顺序；数值风险 = 合法但结果不对；非 UB = 行为明确但慢。

## 10 对拍流程

对拍 = 暴力程序 + 正解 + 随机生成器 + 脚本比较。

### 三个独立程序

1. `brute.cpp`：最直接的方法，保证正确；输入规模要按它的实际复杂度压到能快速跑完。
2. `sol.cpp`：你要提交的版本。
3. `gen.cpp`：生成合法随机小数据，规模要适配暴力；用测试编号作为种子，既能变化又能复现。

### 编译（分别执行）

每改一个文件就要重新编译它：

```
g++ -std=c++14 -O2 -o brute brute.cpp
g++ -std=c++14 -O2 -o sol sol.cpp
g++ -std=c++14 -O2 -o gen gen.cpp
```

### 对拍脚本 `run_test.sh`

```
#!/bin/bash
for i in $(seq 1 1000); do
    ./gen "$i" > in.txt
    ./brute < in.txt > ans.txt
    ./sol < in.txt > out.txt
    if ! diff -q ans.txt out.txt > /dev/null; then
        echo "WA on test $i"; cp in.txt hack.in; exit 1
    fi
done
echo "All passed"
```

流程：`gen` 造数据 → `brute` 跑出答案 → `sol` 跑出结果 → `diff` 对比。有差异则报 WA 并保存输入到 `hack.in`。若程序可能死循环，还应给两次运行加超时。

### 生成器模板 `gen.cpp`

```
#include <cstdlib>
#include <iostream>
#include <random>
int main(int argc, char **argv) {
    unsigned seed = argc > 1
        ? static_cast<unsigned>(std::strtoul(argv[1], nullptr, 10))
        : 42u; // 无参数时仍可复现
    std::mt19937 rng(seed);
    std::uniform_int_distribution<int> n_dist(1, 10);
    std::uniform_int_distribution<int> a_dist(0, 99);
    int n = n_dist(rng);
    std::cout << n << "\n";
    for (int i = 0; i < n; i++)
        std::cout << a_dist(rng)
            << (i+1 == n ? '\n' : ' ');
    return 0;
}
```

模板只负责演示可复现的随机种子；真正对拍时还要按题意加入极值、重复值、结构化数据和无解数据，不能只依赖均匀随机。

## 手造边界

- 最小规模 ( $n = 1$ ) 和最大规模 (但仅能暴力跑的范围)
- 全相等 / 全 0 / 全 1
- 已升序 / 已降序
- 无解输入 ( $-1$  或 NO 的情况)
- 紧贴边界 ( $n = 10^9$  触顶、模数边界等)

## 自查

- 我能在 10 分钟内写好暴力 + 正解 + 生成器 + 脚本吗？
- 我知道为什么要固定种子、存第一个差异到 `hack.in`？
- WA 时我能分段定位哪个环节出错？

## 11 常见缺口

- 历年 CSP-J/NOIP 普及组真题中，模拟和扫描反复出现，但涉及的语言边界、整数范围、输入格式各有不同。
- 常见缺口：
  1. 字符串区间哈希：不会  $O(1)$  比子串，导致树镜像、前缀查询写不顺。
  2. 数学与计数：不习惯贡献计算（把枚举子结构改成统计每个元素的参与次数）；遇到计数题不会建立映射/双射，也不会区分排列与组合、加法原理与乘法原理；一元二次方程的判别式、根式规范化手写不稳。
  3. 复杂度反推：不读测试点表就写，写到 TLE 才发现要优化。
  4. 对拍：写完就跑，不造 hack 数据，不查边界。
- 线性筛、二分三分、Trie、区间哈希、树状数组、单调队列、状态图未必以裸模板出现，但常作为综合题的一部分。

## 创作声明与许可

除另有说明的第三方内容外，本文由 scmmm 原创。本文以 [Creative Commons 署名-非商业性使用 4.0 国际版 \(CC BY-NC 4.0\)](#) 发布。

在注明原作者为 scmmm、保留本许可声明和许可证链接，并标明修改内容的前提下，允许非商业性转载、分享与修改。任何商业性质的使用、转载、修改、发行，或将本文用于商业产品、课程或服务，均须事先获得 scmmm 的授权。

|             |
|-------------|
| 祝各位选手取得理想成绩 |
|-------------|